EdgeShield

Edge Telemetry Monitoring Platform

Project Report

1. What is EdgeShield?

EdgeShield is a practical system we built to keep an eye on computers and environment conditions across multiple locations in real time. At its core, it collects performance data from PCs (CPU, memory, disk, battery) and environmental readings (temperature and humidity) from ESP32 sensors, then stores everything in a structured data lake so we can monitor it through a clean web dashboard.

The idea started simple: we wanted a way to know if machines were struggling or if rooms were getting too hot or humid, without having to walk around and check every device manually. Over time it grew into a full pipeline — from small edge devices all the way up to a Databricks lakehouse and a monitoring website.

2. What Problem Does It Solve?

In most labs, offices, or multi-room setups, people only notice problems after something goes wrong. A computer starts running slow, a machine runs out of disk space, or a room becomes uncomfortably hot. By the time someone notices, work has already been affected.

EdgeShield gives early visibility. You can see which machines are under heavy load, which ones have gone offline, and what the temperature and humidity look like in different rooms — all from one place. It is especially useful when you have multiple blocks or floors and cannot physically check every room regularly.

In short, it helps with:

- Knowing the health of every computer without logging into each one
- Tracking temperature and humidity room by room
- Catching issues early through simple alerts (high CPU, low battery, high temperature, offline devices)
- Having historical data so you can look back and understand patterns

3. Existing Approaches vs EdgeShield

Before building EdgeShield we looked at how people usually handle this kind of monitoring. Most of the common options fall short when you need both computer health and room-level environment data together, especially on a limited budget.

3.1 How things are usually done today

- **Manual checking —** Someone walks around, opens Task Manager or logs into machines one by one. Temperature is checked with a handheld meter or not at all. This works for two or three machines but falls apart as soon as you have more rooms or blocks.
- **Heavy commercial tools —** Products like enterprise monitoring suites (Datadog, Zabbix, PRTG, etc.) are powerful, but they are expensive, often overkill for a lab or small campus, and rarely include simple room temperature/humidity out of the box without extra paid sensors and licences.
- **Separate systems —** People sometimes run one tool for computer metrics and a completely different IoT platform for sensors. The two never talk to each other, so you end up switching between dashboards and never get a single view.
- **Cloud-only solutions —** Many modern tools assume constant internet and paid cloud accounts from day one. They are hard to run on a local network with free or low-cost resources.

3.2 What EdgeShield does differently

We designed EdgeShield around the constraints we actually had: limited money, existing computers, free cloud tier, and the need to cover both machines and rooms.

Aspect	Typical Existing Approach	EdgeShield
Computer monitoring	Enterprise agents or manual checks	Lightweight Python client on each PC
Room temperature & humidity	Separate IoT platform or not tracked	ESP32 + DHT sensor per room, same pipeline
Single combined view	Usually two different dashboards	One dashboard for both PCs and sensors
Cost to start	Licence fees + cloud costs	Only ESP32 + sensors; rest free / existing
Local network first	Often assumes public cloud	Works fully on same Wi-Fi; cloud is storage
Data ownership & structure	Vendor-controlled or scattered files	Clean lakehouse (bronze → silver → gold)
Alerts	Complex rules or paid add-ons	Simple rules now; room for smarter models later
Scaling path	Buy more licences / higher tiers	Same architecture; only cloud capacity changes

The main difference is that EdgeShield treats computers and room sensors as the same kind of thing — just devices that send telemetry. Because everything lands in one lakehouse and one dashboard, you do not have to stitch two systems together. At the same time we kept the cost close to zero by using free Databricks capacity and hardware we already owned.

4. Detailed Breakdown of the Project

The system is made of a few clear layers. Each one does one job and hands the data to the next.

4.1 Edge Clients (PCs)

A lightweight Python program runs on each computer we want to monitor. Every few seconds (currently every 10 seconds) it collects CPU usage, RAM usage, disk usage, battery level and

health, and basic system information. It then sends this data as a JSON payload to the central server over the local network.

We designed the client so that it can start automatically when the machine boots. Once installed, you basically just turn the computer on and it starts reporting. No daily manual steps.

4.2 ESP32 Environmental Sensors

For temperature and humidity we use ESP32 boards with DHT11 or DHT22 sensors. One sensor unit is placed in each room. The ESP32 connects to the same Wi-Fi network as the server, reads the sensor, and posts the readings to the same server endpoint. This keeps the architecture consistent — both PCs and sensors talk to the same place.

4.3 Central Server

The server is a relatively simple Python service (FastAPI-style). It receives all the incoming telemetry from both PCs and ESP32 devices, then inserts the data into Databricks tables. It does not try to do heavy processing itself. Its main job is reliable ingestion.

4.4 Data Lakehouse (Databricks)

We use Databricks as the storage and processing layer. Data lands in a bronze layer (raw), gets cleaned in a silver layer, and is finally organised into a gold layer that is ready for analytics and dashboards. The gold layer has a simple star-schema style design with a device dimension and separate fact tables for CPU, RAM, disk, battery, and environment metrics.

This separation makes it easy to query only what we need and keeps the data clean even when devices send data at slightly different times.

4.5 Web Dashboard

The website sits on top of the gold layer. It lets you select individual devices or look at the whole fleet, choose time ranges (1 hour, 6 hours, 24 hours, 7 days, or a custom range), and view KPI cards plus clean charts. There is also a simple rules-based alert section for high resource usage, offline devices, and temperature thresholds.

The dashboard was built from scratch to stay lightweight and professional. It only talks to the gold layer and does not depend on the rest of the codebase.

5. How to Deploy at Larger Scale

The current setup works well for a small number of machines and rooms. Scaling it to multiple blocks with many rooms follows a straightforward pattern.

5.1 Basic Deployment Pattern

1. **Server machine —** Install and run the server code on one reliable computer that stays on. Note down its local IP address.

2. **Client machines —** Install the client program on every PC you want to monitor. In the client configuration, put the server's IP address. Make sure the client starts automatically on boot.
3. **ESP32 sensors —** Place one temperature + humidity unit in each room. Configure each ESP32 with the same Wi-Fi credentials and the server IP.
4. **Network rule —** All devices (server, PCs, and ESP32s) must be on the same Wi-Fi / local network so they can reach each other.

5.2 Multi-Block / Multi-Room Setup

Imagine a building with several blocks, and each block has multiple rooms. The practical way to handle this is:

- Put one ESP32 + DHT sensor in every room that needs environmental monitoring.
- Install the client on every computer that should be tracked.
- Keep a single central server (or one server per major network segment if the buildings are far apart).
- All devices still report to the same server, which pushes data into Databricks. The dashboard can then filter by device or look at the whole fleet.

Because each device is identified by its own device_id, the system naturally separates rooms and computers even when hundreds of them are reporting.

5.3 Data Flow at Scale

Client / ESP32 → Server (local network) → Databricks (bronze → silver → gold) → Web Dashboard

This flow stays the same whether you have 5 devices or 500. The main change at larger scale is the capacity of the cloud side and the reliability of the server machine.

6. Budget

One of the practical advantages of this project is that most of it can be run with very little money.

What actually costs money:

- ESP32 development boards
- DHT11 or DHT22 temperature and humidity sensors

What we used for free:

- Existing computers for running the client and the server
- Databricks Free Edition for the lakehouse and SQL warehouse
- Open-source software for the entire stack (Python, FastAPI, React, etc.)
- Local Wi-Fi network that was already available

So the only real hardware cost is the ESP32 + sensor units. Everything else was built and run on resources we already had or on free cloud tiers.

7. Future Improvements

7.1 Client Auto-Start

We already aim for the client to start as soon as the computer is turned on. Making this completely reliable across different operating systems (Windows services, Linux systemd, etc.) is still something we can polish further so that no one has to remember to launch anything manually.

7.2 Version 2 – Smarter Alerts and Anomaly Detection

Right now alerts are rule-based (CPU above 85%, temperature above 35°C, device offline, and so on). Once we have collected enough real data from running the system for days or weeks, we can train simple models that notice unusual patterns — for example a machine whose behaviour suddenly changes even if it has not crossed a fixed threshold. This is planned as V2.

7.3 Version 3 – Asking Questions in Normal Language

A later step is to let users type questions such as "Which machines had high memory usage last night?" or "Show me temperature trends in Block B for the last week" and get answers directly from the gold layer data. This would sit on top of the existing dashboard rather than replacing it.

7.4 Other Practical Improvements

- Better handling of network interruptions so clients keep a small local buffer and send data when connectivity returns
- Role-based access on the dashboard if more people start using it
- Simple mobile-friendly view for quick checks

8. What Needs to Be Upgraded When We Scale Further

The free-tier version is enough for development and small deployments. Once the number of computers and rooms grows significantly, a few things will need attention:

- **Cloud database subscription —** Databricks Free Edition has daily usage limits and the SQL warehouse stops after a period of inactivity. For continuous production use we would need a paid plan so the warehouse stays available and can handle higher query volumes.
- **More capable server hardware —** With a large number of clients sending data every 10 seconds, the single server machine will eventually become a bottleneck. Moving it to a more powerful machine or splitting ingestion across a couple of servers would help.
- **Network planning —** If different blocks are on separate networks, we may need a small local collector in each block that forwards data to the central server, or use a VPN / more advanced networking setup.
- **Monitoring the monitors —** At larger scale we should also watch the health of the server itself and the Databricks pipeline so that a failure in the collection layer does not go unnoticed.

9. Closing Thoughts

EdgeShield started as a practical need — knowing what our machines and rooms were doing without constant manual checking. The architecture we ended up with is deliberately simple: small programs on the edge, a single place that receives the data, a proper lakehouse for storage and cleaning, and a focused dashboard on top.

Because the pieces are loosely connected, we can improve one part (better alerts, natural language questions, more sensors) without rewriting everything. The same design also scales from a handful of devices to a multi-block setup, as long as we are ready to move from free cloud resources to paid capacity when the load grows.

Compared with the usual mix of manual checks, expensive enterprise tools, or separate computer and IoT systems, EdgeShield gives a single, low-cost view of both machines and rooms. That combination is what made the project worth building.

The most important next steps are collecting real data for a longer period, making the client completely automatic on boot, and then building the smarter alert layer (V2) on top of that real data.

— End of Report —